

FULL APPLICATION QA AUDIT

Nut AI Complete Quality Report

Every feature audited across three parallel tracks — UI and navigation, AI inference and scan chains, data engine and packages. Every bug, bloat item and UX issue catalogued with evidence, impact and fix. No code changed.

Nut AI Engineering

101 findings · 0 blockers · 10 critical · 43 major · 48 polish

Photo scan · AI assistant · Barcode · Label · Receipt · Logging ·

Workouts · Data

30 SEPTEMBER 2026 · BASELINE 1A711A9 · 733 TESTS GREEN

Table of Contents

1. Executive Summary	2
2. Scope, Method and Severity Model	4
3. P0 - Blockers	5
4. P1 - Critical (10 findings)	5
5. P2 - Major (43 findings)	8
Scan and AI reliability (10)	8
Provider and validation (3)	11
UI and UX (15)	12
Data, packages and tests (15)	16
6. P3 - Minor and Polish (48 findings)	19
Assistant, scan and AI polish (13)	19
UI polish, copy and navigation (15)	21
Data, packages and hygiene (20)	24
7. Security and Privacy Audit	27
Key storage	27
Where keys and data travel	28
Reseller economics	28
8. Performance and Size Audit	28
Bundle and corpora	28
Boot and runtime	29
Token economy	29
9. Bloat Census	29
Dead dependencies and artifacts	29
God files and duplication	30
10. Fix Roadmap	31
Wave 1 - Quick wins (about one working day)	31
Wave 2 - Reseller reliability and honest errors	31
Wave 3 - Trust and consistency	32
Wave 4 - Structural	32

1. Executive Summary

This report is a complete quality audit of the Nut AI application — the Expo/React Native offline nutrition and training tracker with bring-your-own-key AI features — as of commit 1a711a9 on 30 September 2026. It was produced by three parallel read-only audit tracks: one covering every user-facing screen and navigation path, one covering the AI inference and scan chains end to end, and one covering the data engine, packages, tests and repository hygiene. Every finding is catalogued with file-and-line evidence, user impact, a concrete suggested fix and an effort estimate. No code was changed during the audit; the working tree, the 733-test suite and the running web build are exactly as they were at baseline.

101	0	10	43	48
findings catalogued	P0 blockers	P1 critical	P2 major	P3 minor

The overall picture is healthy for an app at this stage. There are no P0 blockers: no launch-crashing defect, no data-loss path found in normal flows, no key leakage to non-provider hosts, and the test suite is green at 733 tests across 87 files. The engines are disciplined — the operation-log/undo design is genuinely good, the 362-dish corpus has zero duplicate names and all 1,451 slots mapped, JPEG metadata stripping is byte-verified before anything is uploaded, and the boot path correctly keeps the 6.2 MB corpus lazy. The TypeScript configuration (strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes) is top-decile, and the TODO census is effectively clean.

The risk concentrates in three places. First, reseller reliability: the app's flagship audience — users of OpenAI-compatible resellers such as aicredits.in — still hits broken paths the official providers never see, including a guaranteed-401 hole in the barcode flow and a streaming fallback that double-bills when a gateway ignores stream:true. Second, silent failure: several of the most important writes and waits (logging a meal, saving weight, waiting on a hung gateway) can fail with no message, no retry and no log, which erodes trust faster than loud bugs do. Third, honest-metrics drift: the corpus's own reports say both '362/362 fully mapped' and '312/362 unusable for deterministic nutrition', and yield verification is at zero — the brand promise needs that gap surfaced, not smoothed over.

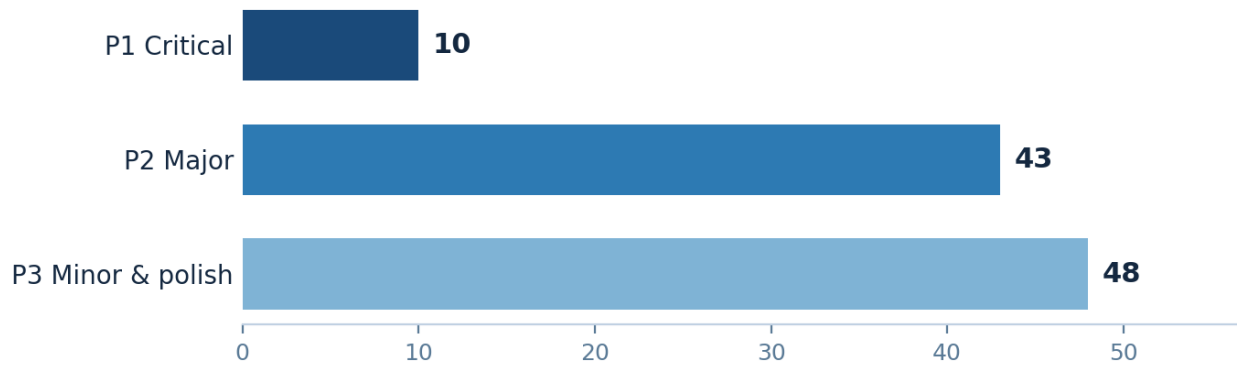


Figure 1: Findings by severity (total 101)

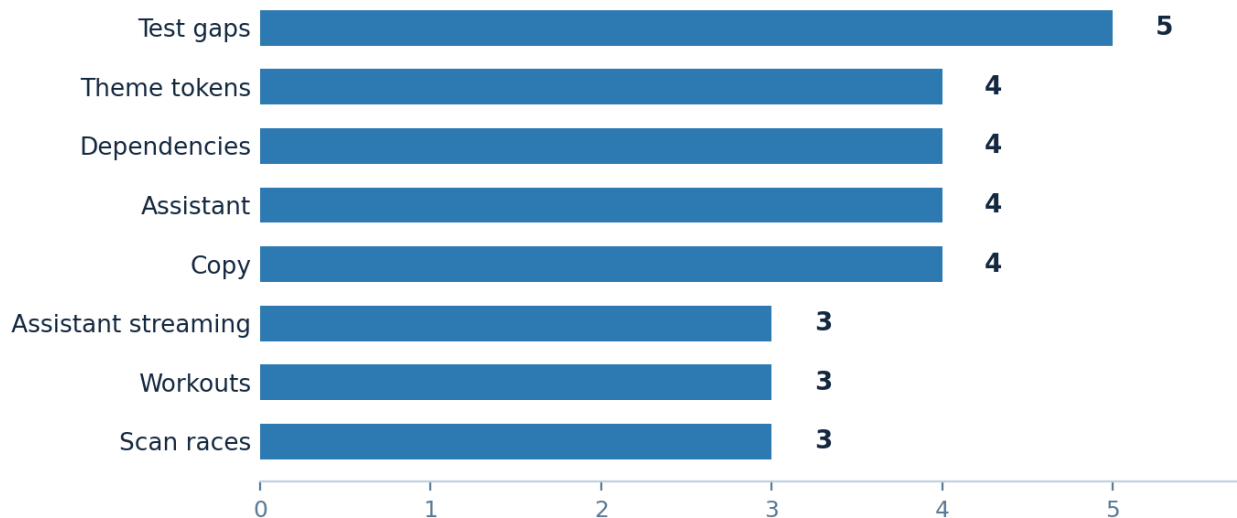


Figure 2: The eight busiest finding areas

The five issues that most deserve the next working session, in order:

- 1. Barcode and 'Other...' lookups bypass the reseller base URL (P1-1)** - Guaranteed 401 for reseller users; a one-line fix per call site.
- 2. Non-SSE streaming fallback double-appends and then double-bills (P1-2, P2-6)** - The correct answer is discarded and a second paid request re-runs it.
- 3. The chat stall guard exists but is never wired (P1-3)** - A silent gateway hangs the assistant forever with no recovery.
- 4. 'Log it' and other critical writes fail silently (P1-8, P2-26)** - The single most important write in the app can no-op without a word.
- 5. Train tab can white-screen on one corrupt row; no ErrorBoundary anywhere (P1-6)** - A cheap guard prevents a whole-feature outage.

Four cross-cutting root causes account for a large share of the 101 findings; fixing each once retires many symptoms:

Async feedback gaps. Eight screens await writes with no catch and no user feedback; one shared pattern (useAction) used everywhere would retire the entire class.

Token-system bypass. tokens.ts promises ESLint enforcement that does not exist, so ~30 hex literals and a dozen off-scale sizes drifted in; adding the two rules plus one token batch closes the whole theme category.

Error-taxonomy compression. Catch-alls collapse distinct failures (offline vs oversized payload vs DB failure vs schema drift) into two user-facing stories; a proper error-kind split fixes diagnosis everywhere at once.

Duplicated logic. Six UI fragments and several helpers exist twice and are already diverging; one consolidation pass removes both the duplication and its future bug supply.

2. Scope, Method and Severity Model

Findings were gathered by three auditors working the same baseline concurrently. Track A opened all 44 screens under apps/mobile/app plus every component and theme file, mapping the full route graph and checking each screen for loading, empty and error states, web-dialog compatibility, theme-token conformance, accessibility and list virtualization. Track B read the AI chains exhaustively — client.ts (1,170 lines) in full, the orchestrator (763 lines) in full, validate, assistant, credentials, base-url, every package in packages/prompt and the schema layer — hunting error-handling gaps, races, streaming faults, model-resolution drift, token waste and key handling. Track C ran the hygiene pass: TODO census, per-file test-coverage mapping, dead-dependency verification by import grep, god files, dead exports, data-integrity spot checks against the built SQLite corpora, and extraction of every acknowledged-but-unfixed debt item from the 181-line engineering worklog.

Severity is user-impact-first. P0 blockers would be launch crashes, unrecoverable data loss, or credential exposure outside the provider — none were found. P1 critical issues break a core flow for a whole user class, silently corrupt or drop user-visible work, or leave a product promise unverifiable. P2 major issues misdiagnose, degrade, over-bill or set recoverable traps; P3 covers polish, consistency and hygiene. Effort is S (minutes to an hour), M (half a day to two days) or L (a structural refactor). Every finding names its evidence location so fixes can be verified against the exact line cited.

Level	Meaning	Count	Action expected
P0	Blocker: crash-on-launch, data loss, key exposure outside providers	0	Immediate hotfix
P1	Critical: core flow broken for a user class, silent loss of visible work	10	Next working session
P2	Major: misdiagnosis, over-billing, recoverable traps, perf debt	43	Scheduled batch
P3	Minor: polish, consistency, hygiene, untested edges	48	Opportunistic

Two environment caveats. The web preview server in the development sandbox is killed by the platform whenever the assistant runs another command, so 'the webapp closed again' during testing was the sandbox reaper, not the app — the build on disk is consistent, and the server is relaunched as the final action of each

session. Separately, reseller gateways behave differently from official APIs in documented ways (aicredits 400s every structured-output request unless the schema is sanitized, and bills image payloads as prompt tokens at roughly ten times their advertised per-scan rate, about Rs 0.49 per scan on gpt-4o-mini); the findings account for those behaviors but credit economics on the reseller side remain outside the app's control.

3. P0 - Blockers

No P0 findings. The audit specifically checked for: crash-on-launch and whole-tab white-screen vectors (one was found — P1-6 — and is graded P1 because it requires a corrupt row to trigger and no user data is lost); unrecoverable data-loss paths in logging, undo and backup (none — the operation-log design and snapshot undo hold); credential exposure to any host other than the configured provider or the user's own reseller (none — withBaseUrl only ever rewrites the api.openai.com prefix, so Anthropic and Google traffic cannot be redirected by a reseller URL); and billing leaks that charge keys without user action beyond the documented cross-provider fallback (found, graded P2-8 because it requires a primary-provider failure first, but it is the closest thing to a P0 in the money category and deserves early fixing).

4. P1 - Critical (10 findings)

Each finding below is independently verifiable at the cited location. Severity reflects current user impact on the supported platforms, not difficulty of the fix - nine of the ten are effort-S once known.

P1-1 · Barcode / web lookup	Barcode and 'Other...' lookups skip the reseller base URL — guaranteed 401 for reseller users
Evidence	orchestrator.ts:459-463 and :749 call runWebLookup(provider, {model, itemName, brand}, credential) WITHOUT the 6th baseUrl argument, while refineMisses, label and receipt paths correctly pass await customProviderBaseUrl().
Impact	For the project's flagship audience — OpenAI-compatible reseller users (aicredits.in, OpenRouter) — the barcode flow and the 'Other...' free-text lookup always hit the official api.openai.com with a reseller key, get a guaranteed 401, and surface as 'Could not find this barcode in the database or online'. A core flow is silently broken for that whole user class, and the diagnosis blames data instead of routing.
Fix	Pass await customProviderBaseUrl() as the final argument in startBarcodeScan and the result-screen 'Other...' lookup. The plain-chat-completions reseller path inside runWebLookup already exists (added in 54263d9) — it is simply not wired at these two call sites. Effort: S

P1-2 · Assistant streaming	Non-SSE response body is double-appended, permanently breaking the 'gateway ignored stream:true' fallback
Evidence	client.ts:1046-1057: the first non-SSE chunk assigns nonSseBody = chunk, but the guard on the NEXT chunk does nonSseBody += chunk and the chunk that SET nonSseBody is immediately appended again — 'AB' becomes 'AAB'/'ABAB'. No test covers the XHR path (client.test.ts only exercises the pure extractFullCompletion).
Impact	Every gateway that ignores stream:true and answers with a plain JSON body fails JSON.parse, which is classified as schema-violation with empty text. The assistant then silently re-requests through the legacy non-streaming chain: double latency, double billing, and the correct first answer is thrown away. This hits exactly the reseller class the streaming feature was built for.
Fix	Append only in the else-branch (or guard 'if (!nonSseBody) return' before concatenating). Add an xhrFactory-driven test that feeds a plain JSON body through runAssistantChatApiStream and asserts a single clean parse. Effort: S
P1-3 · Assistant streaming	Stall guard exists but is never wired; no XHR timeout either
Evidence	client.ts:1008-1024 implements stallTimeoutMs but it is only applied via req.stallTimeoutMs; assistant.tsx:277-280 calls runAssistantChatApiStream WITHOUT passing it, and xhr.timeout is never set.
Impact	A gateway that accepts the connection and then sends nothing hangs the chat indefinitely: the bubble stays on 'Thinking...', loading stays true so send() refuses further input, and the only escape is leaving the screen. This is precisely the frozen-conversation failure the streaming design was meant to prevent.
Fix	Pass stallTimeoutMs: 45_000 from assistant.tsx and optionally reset the timer on every onprogress chunk so mid-stream stalls are also covered. Effort: S
P1-4 · Photo scan	Blanket preprocess catch repeats the 'could not read the photo' mislabel class
Evidence	orchestrator.ts:86-96, 518-523, 622-626: catch { setPhase({kind:'failed', message:'Could not read the photo. Try taking it again.', canRetry:false}) } wraps the entire preprocess stage with no error inspection and no logging.
Impact	ANY preprocessing throw — OOM, RangeError, atob failure, ImageManipulator native crash — is diagnosed as an unreadable photo with retry disabled and zero diagnostics. This is the exact mechanism that mislabeled the thali RangeError bug (fixed in ba96892); it will recur under a different root cause and be equally invisible.
Fix	catch (err) and branch: RangeError / string-length errors get 'this photo is too large to process'; decode errors keep the current copy; transient native errors get canRetry:true. Always log the raw error. Effort: S

P1-5 · Saved foods	'Log again' is a dead control — the screen has zero working actions
Evidence	saved-foods.tsx:61-70 renders the row as a plain with a Log again label; there is no Pressable and no onPress anywhere in the row.
Impact	The screen's entire purpose — one-tap re-logging of a previously saved food — does nothing when tapped. The only interactive-looking element on the screen is inert, making the whole feature an orphan.
Fix	Wrap the row in a Pressable that snapshots the meal and routes to /food-review (mirroring repeatSnapshots in (tabs)/food.tsx), or remove the 'Log again' label until the action is wired. Effort: S

P1-6 · Workouts	Unguarded ProgramInput.parse in render can white-screen the whole Train tab; no ErrorBoundary exists anywhere
Evidence	train.tsx:15: {programs.map(p => { const plan = ProgramInput.parse(JSON.parse(p.definition_json)); ... — a bare parse inside the render map. routines.tsx:343 and programs.tsx:174 wrap the identical parse in try/catch; app/_layout.tsx registers no ErrorBoundary.
Impact	A single corrupt definition_json row (partial migration, hand-edited DB, future schema drift) throws during render and the entire Train tab dies with no recovery UI, because no error boundary exists at any level of the tree.
Fix	Wrap the parse per-row in try/catch and render a 'This program could not be read' card; add a root ErrorBoundary in app/_layout.tsx that offers 'Go home' and keeps the rest of the app alive. Effort: S

P1-7 · Assistant UX	Chat never scrolls to the newest message
Evidence	assistant.tsx:467-520: the conversation is a plain with no ref, no onContentSizeChange handler, no scrollToEnd call, and no maintainVisibleContentPosition.
Impact	When a reply — or a streaming bubble growing word-by-word — extends past the fold, the user stares at a frozen screen with no visible activity. During streaming this reads as 'it hung', which is the exact perception the feature was built to avoid.
Fix	Add a ScrollView ref and onContentSizeChange={() => ref.current?.scrollToEnd({animated: true})} (guarded by reduce-motion), or migrate the conversation to an inverted FlatList which pins the newest bubble natively. Effort: S

P1-8 · Logging	'Log it' failure is silently swallowed
Evidence	result.tsx:185-197: try { ... logMeal(...) ... router.replace } catch { setLogging(false) } — the catch discards the error entirely.
Impact	If the DB write throws, the button un-grays and nothing else happens: no message, no retry, no toast. The user may believe a meal was logged when it was not, or re-scan and log a duplicate; the app's single most important write can no-op without a word.
Fix	Set an error string in the catch and render it above the action bar with a Retry button — the useAction hook already implements this feedback pattern for other actions. Effort: S

P1-9 · Data layer tests	The entire write/undo/settings layer (repo.ts, 780 lines) has zero direct unit tests
Evidence	No repo.test.ts exists anywhere in apps/mobile; grep shows repo.ts appears in no test import list. Every invariant is guarded only by Playwright e2e — the slowest, flakiest layer, and one that npm run check does not even execute.
Impact	logMeal, deleteMeal, updateMealSlot, logWeight, undo/redo and resetEverything carry the app's money-path invariants: append-only goals, snapshot-based undo, idempotency-key dedupe. A regression here corrupts user data silently, and CI would stay green.
Fix	Port the e2e-backed scenarios to vitest against @nutai/db-adapter/node's in-memory DB (repo.ts is already adapter-injectable via openUserDb). Start with logMeal roundtrip, undo/redo chains, and idempotency-key dedupe. Effort: M

P1-10 · Data honesty	Corpus quality reports contradict each other: 312/362 dishes 'unusable for deterministic nutrition' while verification claims 362/362 fully mapped
Evidence	docs/data/indian-dishes.mapping-report.json: fullyResolvable 50, partiallyResolvable 135, dishesUnusableForDeterministicNutrition 312, VERIFIED 0. docs/data/indian-dishes.mapping-verification.json: fullyMappedDishes 362, hardErrorCount 0. Every yieldModel in the seed carries status:'unverified'.
Impact	'Fully mapped' means every slot has a food ID — not that yields or portion grams are verified. For a product whose stated brand is 'never show a number it cannot justify', 86% of dishes run on priors while the headline report says 0 hard errors. The numbers are both true and totally contradictory without reading the code — an honesty-governance gap.
Fix	Make mapping-verification.json also report verifiedYieldDishes (currently implied 0), gate CI on that metric, and surface the fully/partially/unusable split in the app's dish knowledge-base header so internal honesty matches external claims. Effort: M

5. P2 - Major (43 findings)

Scan and AI reliability (10)

P2-1 · Scan error handling	runScan catch-all: RangeError/TypeError become 'offline' with retryable:true
Evidence	client.ts:253-268: catch (err) classifies everything except AbortError as kind:'offline', message 'No connection to the provider.', retryable true — including errors thrown INSIDE the try by JSON.stringify(built.body) on an oversized base64 payload.
Impact	A request that can never succeed (payload too large for the JS engine) is reported as a network outage and the UI invites the user to retry forever, burning time and credits on every attempt.
Fix	Classify err.name RangeError/TypeError as a distinct internal-error kind with its own copy; reserve 'offline' for network-shaped failures (fetch rejection without a response). Effort: S

P2-2 · Scan error handling	runPipeline catch-all: DB failures reported as model shape errors
Evidence	orchestrator.ts:171-200: the whole pipeline stage sits in one try; a DB open/write failure is caught by the same catch as a schema mismatch and set to null, producing 'The model answered in a shape we could not use. This one is on us — try once more.'
Impact	A corrupt or not-yet-open SQLite database shows a schema diagnosis and recommends a retry that deterministically fails, while the real cause is never logged anywhere.
Fix	Split the try blocks (storage open vs pipeline run), log the caught error, and give storage failures their own honest message with no retry suggestion. Effort: S

P2-3 · Fix-result path	runCorrectionIntent has no timeout, no abort, and no fence stripping — parse failure silently triggers a billed re-analysis
Evidence	client.ts:531-607: fetch without AbortController, raw JSON.parse(rawResult) with no markdown-fence handling (unlike postVisionJson at :390-400). result.tsx:558-567 then falls through to a full re-analysis on failure.
Impact	A hung gateway freezes the Fix-Result flow forever (the caller awaits it). A fenced reply throws SyntaxError, is shown as a generic retryable error, and the silent fallback re-runs a fully-billed vision scan the user never asked for.
Fix	Add AbortController + timeout, reuse the fenced-extraction helper, and return schema-violation on parse failure so the UI can offer the re-analysis as an explicit choice. Effort: S

P2-4 · Scan races	Background web-lookup auto-apply clobbers concurrent user edits; stale lookups write into the current scan
Evidence	orchestrator.ts:298-300 auto-applies when a lookup returns exactly one option; store.ts:167-197 applyWebOption overwrites grams/name/snapshot unconditionally. There is no scan epoch — refineMisses is fire-and-forget.
Impact	If the user hand-edited that row's grams while the lookup was in flight, the late response silently overwrites their value. Results from an abandoned previous scan can also land in the current session's phase.
Fix	Tag rows with userEditedAt and skip auto-apply on edited rows; add a monotonically increasing scanId to the phase and ignore web-lookup callbacks from older scans. Effort: M

P2-5 · Scan races	Grams input is uncontrolled: display goes stale under auto-apply, and clearing the field logs 0 g
Evidence	result.tsx:357-363: editGrams(row.id, Number(t))} /> — defaultValue snapshots once; Number("") === 0.
Impact	When applyWebOption changes grams underneath, the displayed value stays stale — the user reviews and logs numbers that differ from stored state. Clearing the input momentarily logs zero grams.
Fix	Make the input controlled on row state (or key it on row.grams) and ignore empty strings in editGrams. Effort: S

P2-6 · Assistant streaming	User abort with zero streamed bytes triggers a second, billed legacy call
Evidence	client.ts:1141-1166 returns {ok:false, text:"", error:{message:'cancelled'}} on abort; assistant.tsx:281-304: the else-branch (no text) fires runAssistantChat — a non-abortable legacy request — because cancelled is not distinguished from empty.
Impact	Leaving the screen or pressing close mid-wait aborts the stream, and the code immediately makes the second API call the user just cancelled; its result writes to unmounted state. The user pays for an answer nobody will read.
Fix	Thread a cancelled flag on the stream outcome and short-circuit before the legacy branch when the outcome was cancelled. Effort: S
P2-7 · Scan races	Scans are uncancellable and can interleave: two analyze loops race setPhase
Evidence	camera.tsx:154-157 fires void startScan(shot.uri) with no handle; orchestrator.ts:118-156 analyze() awaits settings/credentials/runScan with no run-id; a module-level lastCapture is shared. An exception from setting()/loadCredential() escapes the void as an unhandled rejection.
Impact	Back-out-and-rescan while a scan is in flight lets two loops race — last finisher wins and the UI flickers between results. A credentials/settings failure leaves the result screen spinning on 'Preparing...' forever because no timeout state exists.
Fix	Add an epoch counter to the store; wrap the analyze body in try/catch that always lands a failed phase; ignore phases from stale epochs. Effort: M
P2-8 · Provider fallback	Cross-provider chat fallback silently bills the user's OTHER saved provider keys while the header shows the configured model
Evidence	client.ts:619-651 builds fallbacks as filter((p) => p !== req.provider).map((p) => ({provider: p, model: cheapestModel(p).id})); assistant.tsx:252 sets the model line from the configured provider only.
Impact	A 429/5xx on the primary provider makes the app spend the user's money on a DIFFERENT provider's cheapest model — possibly a worse or more expensive-for-them choice — while the UI still displays the configured model and host. 'Shows one model, calls another', plus unconsented spend across accounts.
Fix	Keep fallbacks same-provider only, or display 'answered via @' after a cross-provider answer and gate cross-provider use behind an explicit setting. Effort: M

P2-9 · Cost ledger	computeScanCost returns 0 for unknown model IDs — reseller usage is under-reported
Evidence	providers.ts:255-264: models not in PROVIDER_MODELS (all custom reseller IDs) yield cost 0, silently.
Impact	The cost tracking the app shows the user omits exactly the reseller scans this user class performs, undermining the token-economy transparency promised elsewhere.
Fix	Return null + a 'custom model — cost unknown' flag instead of 0, and render that honestly in the ledger. Effort: S

P2-10 · Scan parsing	extractPayload has no fence stripping on the scan path — one fenced reply loses a billed scan
Evidence	client.ts:146-194: safeParse is a bare JSON.parse(text), while postVisionJson (:390-400) and runWebLookup (:506-516) strip markdown fences and isolate braces.
Impact	On the degraded json_object / instruction-schema retry, a gateway that wraps its answer in ``` fences turns a billed scan into 'The provider returned malformed JSON' with no recovery — while the label, receipt and web paths survive the identical reply.
Fix	Route extractPayload through the same fenced-extraction helper used by the other paths. Effort: S

Provider and validation (3)

P2-1 · Base URL validation	No scheme or /v1 repair: bad pastes produce wrong diagnoses
Evidence	base-url.ts:20-37: normalizeBaseUrl only trims and strips /chat/completions + trailing slashes; withBaseUrl blindly concatenates.
Impact	'aicredits.in/v1' (no scheme) becomes a relative-URL fetch failure shown as 'offline'; 'https://aicredits.in' (missing /v1) 404s and classifies as 'That model is not available on your account'; http:// is accepted silently for a key-bearing request. All three send the user hunting in the wrong place.
Fix	On normalize: require ^https://, warn when /v1 is missing and the host is not localhost; in validate.ts, on reseller 404 include 'check that your base URL ends in /v1' (today that hint fires only when NO base URL is set). Effort: S

P2-2 · Key handling	Google API key is placed in the URL query string on chat and correction paths
Evidence	client.ts:558 and :711 build ...models/{model};generateContent?key={cred}; the scan path correctly uses the x-goog-api-key header (providers.ts:240).
Impact	The key lands in any URL logging — RN network inspectors, corporate proxies, crash breadcrumbs — an exposure inconsistent with the header already used elsewhere in the same file.
Fix	Switch both call sites to the x-goog-api-key header. Effort: S

P2-3 · Prompts	Literal backslash-n bug corrupts the correction prompt's item list
Evidence	packages/prompt/src/correction.ts:14 joins rows with '\n' (the two characters backslash + n), not a newline; client.ts:562 repeats the class with '\\n\\n' between system and user prompts on the Google correction path.
Impact	The correction-intent prompt renders its item list as one mangled line, degrading parse success — which increases the silent fallbacks to billed full re-analysis described above.
Fix	Replace the escaped sequences with real newlines in both files. Effort: S

UI and UX (15)

P2-1 · Scan result UX	The 'ready' view has no Close or Discard — an accidental scan has no way out
Evidence	result.tsx:199-260 and 262-414: the ready-state cards offer only Review / Fix result / Log it; the analyzing and failed states DO have a close button.
Impact	A user who scanned by accident must fall back to system back/gesture (invisible on web), and the draft meal lingers with no explicit discard path.
Fix	Add a small Discard / close affordance in the ready-state header calling reset() + router.back(). Effort: S

P2-2 · Unsaved edits	Hardware-back dirty guards are Android-only — web browser back silently loses edits
Evidence	custom-food.tsx:144-154, recipes.tsx:188-196, routines.tsx:40-50, search.tsx:45-55, exercise-detail.tsx:30-39 all rely on BackHandler.addListener('hardwareBackPress', ...) with no web equivalent.
Impact	On the web build the browser back button — the only back affordance — bypasses the 'Discard changes?' confirm in custom-food and recipes, destroying unsaved work without warning.
Fix	Add a Platform.OS === 'web' branch registering beforeunload (and a history popstate guard) for the two dirty-guard screens that matter most (custom-food, recipes). Effort: M

P2-3 · Web dialogs	DayTimeline ships a second, inferior web-alert shim that bypasses the design system
Evidence	src/components/DayTimeline.tsx:3-19 defines a local showAlert using window.confirm with positional button mapping; used at :133 ('Copy yesterday?') and :138 ('Delete meal?'). The global styled shim (src/ui/alert-web.ts) already covers all 35 other Alert.alert sites via app/_layout.tsx:11.
Impact	The two most destructive confirms in the daily surface render as bare browser dialogs — visually inconsistent, positionally mapped, unqueued and unstyleable — one refactor away from the exact drift the shared shim was built to prevent.
Fix	Delete the local showAlert, call Alert.alert directly (the global shim handles web), and drop the now-dead Platform import. Effort: S

P2-4 · Theme tokens	Screen.tsx hardcodes error red '#ef4444' on the component rendering errors for ~15 screens
Evidence	Screen.tsx:205: {error}. tokens.ts explicitly reserves red for theme.safety and permits hex literals in the tokens file only.
Impact	The same error renders one red in light mode and another in dark mode (safety is #F0655B there), and the violation sits on the highest-traffic error path in the app.
Fix	Use t.safety from the theme object. Effort: S

P2-5 · Theme tokens	Tailwind-palette leftover hexes, including the forbidden amber
Evidence	programs.tsx:191 '#10b98120'; equipment.tsx:193/204/210 '#10b98120'/'#f59e0b20'/'#3b82f620'; routines.tsx:233 '#ffffff0a'; indian-dishes.tsx:165 '#3b82f6'.
Impact	Colors that never underwent dark-mode contrast checking, outside the palette; amber specifically contradicts the design rule that uncertainty is violet, never amber (a deliberate scolding-color ban).
Fix	Map to theme.affirm / theme.uncertainBg / theme.protein tints; add token entries if the tint style is wanted. Effort: S

P2-6 · Theme tokens	'#ccc' border and onboarding chrome hexes bake in light-mode assumptions
Evidence	AssistantCards.tsx:54 borderColor '#ccc'; onboarding/Chrome.tsx:27,44,146 ('#EDEDf0', '#F3F3F6', '#E5E5EA') and onboarding/Controls.tsx:58,89 ('#F3F2F8', '#F0F0F3') each behind a theme.isDark ? token : hex ternary.
Impact	Dark mode is a second-class path in onboarding — contrast never checked, maintenance doubled by every ternary; #ccc is a glaring border on dark backgrounds.
Fix	Add bgSunkenVariant-style tokens and delete the ternaries; replace #ccc with theme.border. Effort: S

P2-7 · Theme tokens	The ESLint hex / tap-target rules promised by tokens.ts do not exist
Evidence	tokens.ts:4-6 claims 'an ESLint rule enforces' the no-hex-literal and MIN_TAP_TARGET invariants; eslint.config.mjs contains no such rule (grep for hex/color/tap/slop: no matches).
Impact	The invariant is documentation-only — which is exactly why roughly 30 hardcoded literals have drifted in across screens. The guardrail exists on paper only.
Fix	Add the two no-restricted-syntax rules to eslint.config.mjs, then fix whatever the first run flags (the findings above are the known inventory). Effort: S

P2-8 · Composer UX	One shared ingredientQuery state serves every unresolved component — inputs type into each other
Evidence	dish-composer.tsx:106 declares a single ingredientQuery/ingredientResults pair; 554-597 binds every unresolved component's TextInput to it, and results render under all of them.
Impact	With two unresolved ingredients, both boxes type into the same query simultaneously and duplicate the same result list; tapping a result attaches to only one row while the other stays stuck.
Fix	Move query/results state into a per-component subcomponent keyed by c.id, mirroring result.tsx's AddIngredientSheet. Effort: M
P2-9 · Accessibility	Sub-44px touch targets without hitSlop across dense controls (MIN_TAP_TARGET violations)
Evidence	food-search.tsx styles.chip ~28px tall (used by 3 chip rows), ingredientResults rows paddingVertical 6 (:862), smallInput minHeight 38; result.tsx:1008-1015 chip minHeight 36; progress.tsx:177 compactChip 40; camera.tsx:400-407 reviewPill 32; dish-composer.tsx:688 paddingVertical 4.
Impact	Miss-taps on the app's densest controls; tokens.ts:173 states small targets MUST set hitSlop, and none of these do.
Fix	Add hitSlop={space.sm} at minimum, or raise to minHeight 44 where layout allows. Effort: M
P2-10 · Accessibility	Icon-only buttons and chip groups missing roles/labels
Evidence	food-search.tsx:963-979 cooking-method chips (the fat chips above them have both); provider-settings.tsx:79-81 header close; result.tsx:922-930 web-lookup submit; recipes.tsx:495,505 edit/delete; result.tsx:324-329 disclosure; log-exercise.tsx:170-183 intensity rows (their redundant track zones have labels — the visible rows do not).
Impact	Screen readers announce unlabelled touchables, making several core flows (method selection, result editing) effectively unnavigable non-visually.
Fix	Add accessibilityRole='button' + accessibilityLabel (+ accessibilityState where selected) to each. Effort: S
P2-11 · Form handling	Unguarded async saves strand the UI on 'Saving...'
Evidence	log-weight.tsx:53-58 (save), edit-goals.tsx:60-72 (save), log-exercise.tsx:147-154/265-293/357-362 (save/add) — all await writes with no try/catch; log-weight also awaits logWeight then router.back().
Impact	Any throw leaves the CTA stuck at 'Saving...' with the form live and no message; the user cannot tell success from failure and may double-submit.
Fix	Wrap in try/catch, show inline error, reset the saving flag — reuse the useAction pattern. Effort: S

P2-12 · Form handling	Optimistic Switches and focus loads without rollback or catch
Evidence	checkin.tsx:60-97: setSafety(updated) then await saveSafety with no try/catch and no revert; (tabs)/profile.tsx:44-77: void (async () => { const handle = await db(); ... })() with no .catch.
Impact	A failed save desyncs the toggle from the database with no signal; a failed profile load renders '—' placeholders forever, indistinguishable from genuinely empty data.
Fix	try/catch + restore previous state + Alert on failure; .catch on the profile loader with a retry affordance. Effort: S

P2-13 · Camera	capture() has try/finally but no catch — the shutter can die silently
Evidence	camera.tsx:140-161: try { await cameraRef.current?.takePictureAsync(...) } finally { setBusy(false) } — a rejection is an unhandled rejection with no user feedback.
Impact	If the camera errors (permission revocation, native crash), the shutter appears dead and the user retries blind with no diagnosis.
Fix	Add a catch that surfaces 'Could not take the photo — try again' on the dark overlay. Effort: S

P2-14 · Empty states	Recent/Frequent lists on the Food tab render blank for new users
Evidence	(tabs)/food.tsx:71-85: foods.slice(0,8).map(...) renders nothing when the lists are empty (only Saved/Favorites draw DB rows).
Impact	A brand-new user sees mode chips with dead space below and no guidance on what this surface will become.
Fix	Render 'Nothing here yet — log a few foods and they will appear.' like saved-foods.tsx:52-59 already does. Effort: S

P2-15 · Duplication	Six near-identical UI fragments drifting apart
Evidence	'Create custom ingredient' mini-form duplicated (food-search.tsx:874-912 vs dish-composer.tsx:582-595, different error UX); fat+method chip rows duplicated (food-search:914-981 vs dish-composer:603-656, the copy already lost its all y props); six local Field TextInput wrappers (Screen.tsx:137, food-review.tsx:193 and meal-detail.tsx:273 where 'onChange' silently means onChangeText, custom-food.tsx:304, recipes.tsx:522); resolveCorpusSelection exists twice in the SAME file (result.tsx:103-124 vs 678-698); describeOperation duplicated (result.tsx:622-639 vs assistant.tsx:538-551); MealProposalCard and WorkoutRoutineProposalCard are one card with different lists (AssistantCards.tsx:62-115).
Impact	Every copy is one refactor away from diverging further — the food-search method chips already lost accessibility props that dish-composer's copy retains; the Field variant with the misleading prop name is a standing trap.
Fix	Extract NewIngredientForm, FatMethodChips, and a single Field into src/components; dedupe the two resolvers and describeOperation; merge the proposal-card pair. Effort: M

Data, packages and tests (15)

P2-1 · Performance	ActiveWorkout polls the full database every second, forever, plus a duplicate timer in workout.tsx
Evidence	ActiveWorkout.tsx:8: setInterval(refresh, 1000) where refresh runs db().then(activeWorkout) — on every mounted tab; workout.tsx:15 runs its own 1s timer on top.
Impact	Roughly 86k DB round-trips per day per mounted layout to update a minute counter and rest seconds — battery drain and jank on low-end Android, duplicated work by design.
Fix	Poll at 1s only while rest > 0 (else 30s), or compute elapsed from a local timer and refresh from the DB only on focus/mutation. Effort: S
P2-2 · Performance	Up to 500 dish rows rendered in a plain ScrollView; Train history unbounded and unvirtualized
Evidence	indian-dishes.tsx:69 LIMIT 500 + :160-170 dishes.map(d =>) inside Screen's ScrollView; (tabs)/train.tsx:16 history.map(...) with no LIMIT on the query.
Impact	All 500 rows mount immediately on web and low-end Android — slow open, high memory; the Train tab mounts hundreds of cards after months of use and re-renders the whole list on any action.
Fix	FlatList with keyExtractor for both (or slice + 'See all' for history). Effort: S
P2-3 · Repo hygiene	Compiled build artifacts are git-tracked inside src/data
Evidence	git ls-files: src/data/backup-core.js, backup-core.js.map, backup-core.d.ts.map tracked alongside backup-core.ts; backup.ts:17 imports './backup-core'.
Impact	Two sources of truth for backup logic — the bundler resolves .ts while node scripts may resolve the stale .js; drift is invisible until a tool picks the wrong one.
Fix	gitignore src/data/backup-core.js*; if a plain-JS copy is genuinely needed for web export, generate it in a build step. Effort: S
P2-4 · Dependencies	Dead dependencies and 2.5 MB of dead wasm shipped in public/
Evidence	package.json declares sql.js and wa-sqlite (zero imports anywhere); public/ ships sqlite3.wasm (0.83 MB), wa-sqlite-async.wasm (1.09 MB), wa-sqlite.wasm (0.59 MB) and test-wasm.html — the web adapter actually uses @sqlite.org/sqlite-wasm (expo-adapter.web.ts:2). @expo/ui and expo-linking are likewise declared and never imported.
Impact	Install bloat, upgrade surface for unused packages, a debug page (test-wasm.html) exposed in the web bundle, and 2.5 MB of dead weight served with every deployment.
Fix	Remove the four dependency entries and the four public/ files; keep @sqlite.org/sqlite-wasm only. Effort: S

P2-5 · Data growth	Undo-journal compaction is dead code — the operations table grows unbounded for the install's lifetime
Evidence	repo.ts:773-780 defines compactHistory; no screen or init path calls it. compactOperations is referenced only in operations.ts and its tests.
Impact	operations rows (with full prevJson/newJson meal snapshots) accumulate forever — the growth risk the backup docs acknowledge is actually unmitigated on-device, on top of the unbounded queries below.
Fix	Call compactHistory (e.g. maxCount 500 / maxAge 90d) inside db() init or after successful undo. Effort: S
P2-6 · Queries	Unbounded hot-path queries: streak scan on every Home focus, mealsForDay N+1
Evidence	(tabs)/index.tsx:64-66: 'SELECT DISTINCT local_date FROM meals WHERE deleted_at IS NULL ORDER BY local_date DESC' — no LIMIT, inside loadData → useFocusEffect; repo.ts:507-544: one log_items query per meal (5 meals/day = 6 queries per focus).
Impact	The hottest screen pays linearly-growing query cost on every focus, plus N+1 latency — harmless at month 3, sluggish at year 3, worst on slow devices and web-wasm.
Fix	LIMIT the streak query (120 covers what is rendered) or derive from day_summaries; single WHERE meal_id IN (...) join grouped in JS. Effort: S
P2-7 · Web adapter	FTS5 regex polyfill silently rewrites the corpus schema on web — a landmine with no test
Evidence	src/db/expo-adapter.web.ts:31-38: if (sql.includes('USING fts5')) rewrites CREATE VIRTUAL TABLE into a plain CREATE TABLE IF NOT EXISTS.
Impact	If @sqlite.org/sqlite-wasm ever ships without FTS5, the corpus schema silently changes shape and every food_fts MATCH query (usda-source.ts:26, ifct-source.ts:26) starts throwing at runtime — currently inert, undiscoverable by tests.
Fix	Delete the polyfill and fail fast with a clear error when FTS5 is missing, or feature-detect and gate. Effort: S
P2-8 · Boot path	Home first paint gated behind migrate + exercise seeding that re-runs every launch
Evidence	repo.ts:51-54: db() awaits migrate(handle) then seedExercises(handle) (which loops the whole EXERCISE_LIBRARY in one transaction, repository.ts:40-43) before any query; (tabs)/index.tsx:89 shows 'Loading your day...' until db() plus 3 queries resolve.
Impact	Cold-start content latency is dominated by first-open DB work (the reported slow web boot lives here plus the web wasm fetch); seeding is idempotent but pays full cost every launch.
Fix	Short-circuit seedExercises behind SELECT COUNT(*) FROM exercises; render the Home shell with skeleton before db() resolves. Effort: M

P2-9 · Test gaps	orchestrator.ts — 763 lines coordinating every scan mode — has zero direct tests
Evidence	src/scan holds only photo-privacy.test.ts, store.test.ts and quick-mode.test.ts; analyze (119 lines), startBarcodeScan (107), startLabelScan (119), startReceiptScan (119) have no contract tests.
Impact	The highest-risk glue code in the app — mode routing, corpus gating, fallback ordering — regresses invisibly; the reseller base-URL miss (P1) is exactly the class of bug a contract test would have caught.
Fix	Extract the pure per-scan decision logic (mode routing, corpus-info gating) into a testable module; add contract tests with a mocked client. Effort: M

P2-10 · Data integrity	Avial's added-fat slot maps to coconut milk (197 kcal/100g) — the warning is generated but nothing consumes it
Evidence	docs/data/indian-dishes.mapping-verification.json sanityWarnings[0]: 'dish:in:avial slot added_fat: usda:170173 (Nuts, coconut milk, canned) has 197 kcal/100g — low for a fat slot'.
Impact	One dish computes its fat from milk calories instead of coconut oil (~900 kcal/100g) — a large systematic undercount; the audit tooling did its job and the result was never acted on.
Fix	Manually override the slot to an oil row; add a CI-fail threshold for fat-slot kcal < 400 instead of a warning. Effort: S

P2-11 · Data integrity	The entire IngredientSuggestion mechanism is provably inert
Evidence	tools/indian-dishes/dish-ingredient-suggestions.json is '{}' (3 bytes); the built nutrition.db contains ZERO dishes with ingredientSuggestions (verified by SQL query); build-sqlite.mjs:137 short-circuits when provenance.recordStatus === 'CURATED' — and all 362 dishes are CURATED.
Impact	A three-file pipeline (generator + injection + dish-composer pre-seeding) is kept alive across the codebase while demonstrably producing nothing — dead machinery that misleads contributors.
Fix	Either delete the mechanism or fix the condition to inject for CURATED dishes whose optional slots are unmapped, then regenerate suggestions. Effort: M

P2-12 · Build tooling	build-sqlite.mjs regex-scrapes the schema from TypeScript source and falls back to a fossil seed
Evidence	tools/indian-dishes/build-sqlite.mjs:77,98: new RegExp('export const ' + name + ' = `...'`).exec(src) over packages/db-adapter/src/schema.ts; the catch-fallback reads 'docs/indian-dishes.seed.v0.1(1).json' — a download-duplicate filename.
Impact	Renaming a schema const silently builds an empty dish knowledge base; the fallback silently rebuilds from a fossil, masking the failure.
Fix	Import the compiled @nutai/db-adapter exports instead of scraping source; make the fallback throw. Effort: S

P2-13 · Test gaps	Two e2e journeys permanently parked as test.fixme; e2e still outside npm run check
Evidence	e2e/wave1.spec.ts:45-71: test.fixme for the 56-day DayStrip history and the lb→kg set-editor conversion; package.json:22 check does not include Playwright e2e (CI-only via web-e2e.yml).
Impact	Two acknowledged product gaps have no active guard, and the end-to-end layer that guards the money paths never runs in the local gate developers actually execute.
Fix	Convert fixme items to ticketed skips or implement fixtures (qa-backup.json exists for seeding); add a lightweight e2e smoke to check. Effort: M

P2-14 · Type safety	as-any casts on parsed-or-raw exercise rows — schema drift risk on user data paths
Evidence	packages/training/src/repository.ts:55,63 cast rows via 'as any' after parsing; eslint.config.mjs:19 disables no-explicit-any repo-wide (18 non-test occurrences, concentrated at the network/wasm boundary which docs accept).
Impact	A schema change in the exercise seed flows through untyped and surfaces as runtime weirdness in workouts rather than a compile error.
Fix	Narrow-unknown parse in the repository casts; re-enable no-explicit-any in warn mode to prevent regrowth. Effort: S

P2-15 · Query bounds	custom-foods full-list SELECT * unbounded
Evidence	src/data/custom-foods.ts:92: SELECT * FROM user_foods WHERE deleted_at IS NULL ORDER BY lower(name), id — no LIMIT, feeds the saved-foods screen.
Impact	Acceptable at personal-food scale today; becomes the next unbounded hot query as the list grows.
Fix	Add LIMIT plus the search filter once the list grows. Effort: S

6. P3 - Minor and Polish (48 findings)

Compact workups: evidence, impact, fix and effort in a single block per finding. Several entries record verified-clean strengths so the census is complete rather than uniformly negative.

Assistant, scan and AI polish (13)

P3-1 · Assistant	runCorrectionIntent mislabels JSON/parse errors as retryable via a 'fetch' string heuristic
Detail	Evidence - client.ts:579-606 inspects err.message?.includes('fetch') to classify. Impact - Parse failures from a bad model answer look like network hiccups and invite pointless retries. Fix - Classify on error type, not message sniffing. Effort: S

P3-2 · Assistant	Empty streaming bubble left on screen when the legacy fallback path throws
Detail	Evidence - assistant.tsx:332-333: the placeholder bubble added at :257 is not removed when the legacy chain errors before producing text. Impact - A dangling empty bubble remains in the conversation after a failure. Fix - Remove the placeholder in the legacy error path. Effort: S
P3-3 · Assistant	256KB stream cap counts UTF-16 code units, not bytes
Detail	Evidence - client.ts:1041-1045 measures responseText.length. Impact - A legitimately long answer near the cap is aborted into "The answer was too long to receive." — benign at max_tokens 2048 but a wrong-world limit. Fix - Measure bytes (Blob.size or TextEncoder) or raise the cap. Effort: S
P3-4 · Scan result	applyIntent captures rows once before its loop — stale after removeRow
Detail	Evidence - result.tsx:574-611 reads rows up-front, then mutates inside the loop. Impact - Removing a row mid-correction can apply operations against a stale snapshot. Fix - Re-read rows per iteration or operate on ids. Effort: S
P3-5 · Confidence	Confidence-chip bands misalign after removeRow
Detail	Evidence - packages/pipeline/src/index.ts:375-379 computes bands against a row list that store.ts:112-120 has already re-indexed. Impact - After deleting a row, the highlighted-confidence row can point one slot off. Fix - Recompute bands from post-removal indices. Effort: S
P3-6 · Credentials	looksPlausible accepts anything length >= 20
Detail	Evidence - credentials.ts:94-96 trims and checks length only (documented trade-off; the provider remains the authority). Impact - Some short vendor tokens are rejected client-side before the provider can judge them. Fix - Document per-provider minimums or delegate entirely to the verify call. Effort: S
P3-7 · Cost lookup	Model-ID comparisons are case-sensitive
Detail	Evidence - PROVIDER_MODELS.find(m => m.id === modelId) in cost/label paths; a 'GPT-4O-MINI' paste still requests fine but loses cost lookup. Impact - Custom-case IDs silently report zero cost (compounding the computeScanCost gap). Fix - Case-insensitive catalogue matching. Effort: S

P3-8 · Token economy	~4.5K-token scan system prompt re-sent verbatim on every scan; no provider prompt caching
Detail	Evidence - packages/prompt/src/system-prompt.ts:42-303 ships a constant SYSTEM_PROMPT per request; no cache_control (Anthropic) or cached-prefix arrangement in providers.ts. Impact - At gpt-4o-mini prices that is roughly \$0.0007/scan of byte-identical repeated input — trivially cacheable. Fix - Add Anthropic cache_control on the system block; keep OpenAI/Gemini prefixes stable (they already are). Effort: S
P3-9 · Web parity	Web blob: URIs into ImageManipulator are an untested platform combination
Detail	Evidence - camera.tsx:246-267 uses the file picker on web with base64:false into the same orchestrator path. Impact - A preprocess failure on web lands in the blanket 'could not read the photo' catch with no web-specific diagnosis. Fix - Add one web fixture test through the preprocess stage. Effort: M
P3-10 · Receipt	Unresolved receipt items are dropped silently when SOME rows resolve
Detail	Evidence - orchestrator.ts:724-732 only names unresolved items in the all-fail case. Impact - Partial receipts quietly lose line items the user photographed. Fix - Surface a muted '2 items could not be matched' line with the names. Effort: S
P3-11 · Receipt	Multi-match receipt items auto-choose option[0] without surfacing the question
Detail	Evidence - Only items with >1 option get a question card; the first option is applied otherwise. Impact - The user never sees that 'paneer' resolved to a specific branded product. Fix - Show the choice card whenever options.length > 1. Effort: S
P3-12 · Performance	SetValues.parse + JSON.stringify recomputed per render for every set row
Detail	Evidence - workout.tsx:53 useEffect deps recompute parses on each render. Impact - Avoidable JS-thread work in the hottest editing surface. Fix - Memoize parsed values per row id. Effort: S
P3-13 · Assistant	Chat rendered as .map over ScrollView
Detail	Evidence - assistant.tsx:467-520; fine at the 40-turn cap, but each bubble holds growing streaming text. Impact - Compounds the no-autoscroll issue; inverted FlatList fixes both in one move. Fix - Migrate to inverted FlatList. Effort: M

UI polish, copy and navigation (15)

P3-1 · Navigation	Invalid/missing meal id renders 'Loading meal...' forever
Detail	Evidence - meal-detail.tsx:49-71,130-136: no else branch when getLoggedMeal resolves null. Impact - A deep link with a bad id is an eternal spinner (exercise-detail.tsx:251-256 handles the same case). Fix - Track notFound and render an error card with Back. Effort: S
P3-2 · Navigation	Assistant close falls back to router.replace('/') — Home state resets
Detail	Evidence - assistant.tsx:160-171: replace kills the back stack when canDismiss() is false; assistant is not declared as a modal in _layout.tsx. Impact - Landing on the assistant cold then closing resets tab/scroll state. Fix - Push assistant as an explicit modal, or navigate to '/(tabs)' instead of replace. Effort: S
P3-3 · Copy	Inconsistent button capitalization: Title Case vs sentence case
Detail	Evidence - 'Create New Routine' (routines.tsx:213), 'Review & Log Dish' (food-search.tsx:1036), 'Log Household Variant', 'Accept New Target' vs 'Log food', 'Start quick workout', 'Save routine'. Impact - Small but pervasive polish signal on primary CTAs. Fix - Adopt sentence case (the dominant convention). Effort: S
P3-4 · Copy	Emoji still shipped despite the Icon system built to replace them
Detail	Evidence - indian-dishes.tsx:166, food-search.tsx:697,745, dish-composer.tsx:530,542, AssistantCards — Icon.tsx:7-10 docstring says the system exists to replace emoji. Impact - Mixed rendering across platforms and inconsistent visual language. Fix - Swap for Icon glyphs. Effort: S
P3-5 · Copy	Vague or dead-end copy in failure states
Detail	Evidence - food-review.tsx:35 'Data corrupted, go back'; log-exercise.tsx:286 'The estimate failed.'; AssistantCards 'X FAILED (Tap to retry)' — but the banner is not tappable. Impact - No reason, no path forward; the retry instruction points at a non-interactive element. Fix - State the cause and the actual button ('Tap Review & Save to try again') or make the banner Pressable. Effort: S
P3-6 · Display	Hero kcal number is unrounded
Detail	Evidence - result.tsx:211,285 print result.totals.kcal directly — 412.40000000000005 is possible; every other screen rounds. Impact - A floating-point artifact on the single most visible number in the app. Fix - Math.round at render. Effort: S
P3-7 · Copy	Search hint says 3-character minimum but the effect fires at 2
Detail	Evidence - food-search.tsx:718-722 instructs 'need at least 3 characters'; :171-172 gates at length >= 2. Impact - Off-by-one instruction that makes the hint look wrong. Fix - Align the gate with the copy. Effort: S

P3-8 · Inputs	Date fields are default-keyboard text inputs
Detail	Evidence - food-review.tsx:170, meal-detail.tsx:156: keyboardType 'default' invites autocomplete on web; validation exists but no inputMode/placeholder format enforcement. Impact - Autocomplete popovers over date entry; avoidable typos. Fix - Set inputMode numeric + explicit placeholder format. Effort: S
P3-9 · Inputs	autoCapitalize keyed on label text heuristic
Detail	Evidence - custom-food.tsx:330: autoCapitalize={label.includes('Barcode') ? 'none' : 'sentences'}. Impact - Rewording the label silently changes keyboard behavior. Fix - Pass an explicit capitalize prop per field. Effort: S
P3-10 · Settings	Custom chatbot-model field displays " when the stored id is a catalogue entry
Detail	Evidence - provider-settings.tsx:290-299: value={assistantModelId && !PROVIDER_MODELS.some(...) ? assistantModelId : ""}. Impact - The field visually disagrees with state; typing replaces rather than appends after re-entry. Fix - Always show the stored id in the field and let the picker rows show selection. Effort: S
P3-11 · Layout scale	Magic numbers where tokens exist
Detail	Evidence - Screen.tsx:38-41 padding space.lg+4 / paddingBottom 180 / gap space.md+2, borderRadius radius.md+2 (:117); ad-hoc fontSizes 15/17/22/23/26/30/34/46 vs the type scale; bottom paddings 80/120/140/150/160/180 across screens. Impact - Off-scale spacing drifts visually and defeats the token system's purpose. Fix - Snap to the scale; add the missing steps to tokens if genuinely needed. Effort: M
P3-12 · Layout scale	dish-composer unique off-scale radius and cramped inputs
Detail	Evidence - dish-composer.tsx:688-689 borderRadius 4, paddingVertical 4. Impact - The only 4px-radius surface in the app; inputs feel pinched. Fix - radius.md and space tokens. Effort: S
P3-13 · Console noise	console.warn noise on fresh installs
Detail	Evidence - indian-dishes.tsx:94,109 warn on expected first-run paths; dish-composer.tsx:488, assistant.tsx:426, profile.tsx:305 are deliberate diagnostics. Impact - Log noise on normal flows; masks genuinely unexpected warnings. Fix - Silence the expected-path warns or gate behind debug flag. Effort: S
P3-14 · Tab bar	Two FAB actions share the same icon; 13-line product comment lives in code
Detail	Evidence - (tabs)/_layout.tsx:42,44 'AI Assistant' and 'Food Database' both use the search icon; :10-22 is a 13-line rationale block. Impact - Icon ambiguity in the quick-action sheet; product rationale belongs in docs. Fix - Distinct icons; move the comment to docs. Effort: S

P3-15 · Accessibility	Misc a11y gaps: disclosures and text inputs without labels
Detail	Evidence - result.tsx:324-329 change-disclosure without role; log-exercise.tsx:170-183 intensity anchor rows unlabelled; indian-dishes.tsx:148-154 and food-search.tsx:758-764 dish-name TextInput lacks accessibilityLabel. Impact - Non-visual users miss context on several editing surfaces. Fix - Add roles/labels. Effort: S

Data, packages and hygiene (20)

P3-1 · Repo hygiene	Layout re-exports an unused constant
Detail	Evidence - app/_layout.tsx:13-14 exports ONBOARDING_DONE_KEY used by no importer. Impact - Odd public surface on the root layout. Fix - Remove the re-export. Effort: S
P3-2 · Hygiene	TODO census is effectively clean — one template-text match
Detail	Evidence - Repo-wide grep: the only TODO/XXX marker is docs/adr/rewrite_adrs.py:559 (ADR filename template). Impact - None — worth recording as a strength; consider making it a convention. Fix - Optionally document the no-todo rule. Effort: S
P3-3 · Test gaps	packages/prompt: 10 of 14 modules have no direct tests
Detail	Evidence - Gaps: providers.ts, system-prompt.ts (303 L), vision-json.ts (169 L), web-lookup.ts (129 L), local-signals.ts, assistant.ts, correction.ts, label-scan.ts, receipt-scan.ts. Impact - Prompt strings drift silently; only wire-transforms/schema-compat have coverage. Fix - Golden-string snapshot tests for system-prompt + request builders. Effort: M
P3-4 · Test gaps	Per-module edge tests missing across engines
Detail	Evidence - gram-engine yields/density/volume/priors/reconcile/bands, resolver gtin.ts (102 L, checksum logic), query/scoring, analytics body/nutrition/reports/training/units — all tested only via index-level suites. Impact - Edge cases (e.g. GTIN check-digit variants) can rot unnoticed. Fix - Cheap per-module unit files for gtin and gram-engine first. Effort: S
P3-5 · Test gaps	Onboarding persistence and dish-portion math have zero direct tests
Detail	Evidence - Gaps: onboarding/persist.ts (163 L), flow.ts, store.ts, data/manual-food.ts (147 L), workout-load.ts, routine-draft.ts, indian-dishes clarification/household/inheritance/portions. Impact - First-run persistence and household-portion math are logic-bearing; failures hit first-run UX and dish accuracy. Fix - Prioritize persist.ts and household.ts. Effort: S

P3-6 · Dead code	Dead exports: isUpToDate, latestUndoableMealOperation
Detail	Evidence - db-adapter/migrate.ts:75 and repo.ts:763 are referenced nowhere outside their defining files (compactHistory handled at P2). Impact - False API surface; compactHistory's siblings suggest intent that was never wired. Fix - Delete or wire. Effort: S
P3-7 · Duplication	Duplicated helpers: clamp x3, normalizeText re-implementation
Detail	Evidence - gram-engine/src/reconcile.ts:75 clampGrams; onboarding/Controls.tsx:196,296,409 inline clamps; packages/resolver/src/scoring.ts:115 normalizeText re-implemented in query/aliases layers. Impact - Divergence risk in numeric clamping semantics across surfaces. Fix - Funnel through shared helpers where semantics match. Effort: S
P3-8 · Dependencies	Phantom zod dependency in prompt package tests
Detail	Evidence - packages/prompt tests import zod directly (wire-transforms.test.ts:115 etc.) but package.json declares only @nutai/core-schema. Impact - Works via hoisting only; breaks under strict workspace isolation. Fix - Add zod to prompt devDependencies. Effort: S
P3-9 · Dependencies	recipe-engine is the odd package with its own devDeps (vitest ^1 conflicts with root ^3)
Detail	Evidence - Only packages/recipe-engine declares typescript+vitest devDeps; 17 siblings declare none. Impact - Version skew risk and inconsistency. Fix - Drop recipe-engine devDeps to match the rest. Effort: S
P3-10 · Data layer	Pragmas asymmetric: native adapter lacks WAL
Detail	Evidence - node adapter sets journal_mode=WAL + foreign_keys (db-adapter/node.ts); native expo-adapter.ts:53 sets only foreign_keys; web sets foreign_keys only. Impact - Native user.db misses WAL write-concurrency benefits — low impact single-connection, trivially aligned. Fix - PRAGMA journal_mode = WAL in openUserDb native path. Effort: S
P3-11 · Web adapter	Every web run() costs two extra statements
Detail	Evidence - expo-adapter.web.ts:60-65: SELECT changes() + SELECT last_insert_rowid() per write, including inside the BEGIN/COMMIT shim. Impact - Doubles statement count for web writes; fine at current volume, worth knowing in profiling. Fix - Use exec callback metadata if the wasm exposes it, or batch. Effort: S

P3-12 · Data honesty	1-4 kcal items render '0 kcal' rather than '<5'; missing energy renders 0 via ?? 0
Detail	Evidence - packages/totals/src/index.ts:186-188 labelRounding calories: kcal < 5 returns 0; packages/pipeline/src/index.ts:110 kcal: food.energyKcal ?? 0 while fiber/sugar/sodium handle null honestly at 114-118. Impact - Label-rounding to zero is legally standard, but '?? 0' for missing energy is weaker than the null handling right below it — a small honesty tension with the brand. Fix - Render '<5 kcal' for (0,5); keep null for missing energy like the micros. Effort: S
P3-13 · Type safety	as-any census: 18 non-test occurrences, worst concentration in client.ts (7)
Detail	Evidence - client.ts:289,560,561,564,570,635,645; assistant.tsx x2; expo-adapter.web.ts x2; search/dish-composer/indian-dishes/recipe-source:64/dish-kb-source:133 one each; eslint no-explicit-any off. Impact - Network-boundary casts are documented-acceptable; the risk is regrowth unflagged. Fix - Warn-mode lint to stop regrowth. Effort: S
P3-14 · Type safety	@ts-ignore usage is exemplary — 4 occurrences, all justified in tests
Detail	Evidence - backup-core.test.ts:351,387,418, shortcuts.test.ts:144 all carry justification comments; tsconfig strict + noUncheckedIndexedAccess + exactOptionalPropertyTypes. Impact - None — record as a strength. Fix - None. Effort: S
P3-15 · Config	Provider constants scattered across 3+ files; UI copy coupled to one reseller
Detail	Evidence - OPENAI_PREFIX (base-url.ts:17), endpoint URLs inline in client.ts, provider list re-hardcoded in repo.ts:87, 'https://aicredits.in/v1' placeholder in provider-settings.tsx:229. Impact - Adding a provider touches several files; the branded placeholder couples UI to one vendor. Fix - Single providers constants module (ids, names, default models, placeholder). Effort: S
P3-16 · Config	Magic timeouts scattered without cross-reference
Detail	Evidence - DEFAULT_TIMEOUT_MS 45s (client.ts:122), REQUEST_TIMEOUT_MS 8s (off-source.ts:3), streaming stall guard 45s (client.ts). Impact - Provider-dependent tuning with no single source of truth. Fix - One timeouts.ts in packages/prompt with named constants. Effort: S
P3-17 · Dependencies	expo-crypto has no direct imports — probably removable
Detail	Evidence - Zero direct source imports; may be transitively required by expo-secure-store/uuid paths. Impact - Possible install bloat. Fix - Verify with expo-doctor, then remove. Effort: S

P3-18 · Repo hygiene	One-off scripts mixed with live gates
Detail	Evidence - scripts/report-before-after.mjs (execSync git diff), fix-incoherent-yields.mjs, add-missing-fillings.mjs are already-applied patches sitting beside the three scripts actually wired into npm run check. Impact - Contributors cannot tell re-runnable gates from historical postmortems. Fix - Move to scripts/oneoff/ with a README line, or delete. Effort: S
P3-19 · Workouts	Program library pages also duplicate the pattern-trap seen elsewhere
Detail	Evidence - routines.tsx:40-50 and search.tsx:45-55 duplicate the Android-only BackHandler dirty guard pattern (covered at P2 for custom-food/recipes). Impact - Same web data-loss exposure, lower-traffic screens. Fix - Include in the same web beforeuards fix batch. Effort: S
P3-20 · Workouts	Workout history streak/day queries fine now; DayStrip 56-day product gap parked
Detail	Evidence - Matches the e2e test.fixme at wave1.spec.ts:45; no active guard for the 56-day history display. Impact - Acknowledged product gap with no regression protection. Fix - Ticket or implement with the qa-backup.json fixture. Effort: M

7. Security and Privacy Audit

Key storage

API keys are stored per provider. On iOS and Android the credential goes into SecureStore with WHEN_UNLOCKED_THIS_DEVICE_ONLY, one entry per provider — the correct primitive. On web the key lives in localStorage (credentials.web.ts:68-80, documented in-file), which any same-origin script can read; the promotion path purges the sessionStorage copy correctly. The web build serves with COOP/COEP headers, which limits cross-origin scripting exposure, but localStorage keys remain readable to any injected same-origin code, so web XSS is the one real key-exposure surface. No key is ever embedded in an error string: the UI shows maskCredential output (first seven, last four), and validation detail ships only status plus a body snippet.

Surface	Native (iOS/Android)	Web
Key at rest	SecureStore, WHEN_UNLOCKED_THIS_DEVICE_ONLY, one entry per provider	localStorage (documented in-file); sessionStorage purged on promotion
Key in memory	Held only in credential objects; never stringified into logs	Same; maskCredential (first 7 / last 4) for all UI display

Surface	Native (iOS/Android)	Web
Main exposure	Device unlock requirement	Same-origin XSS can read localStorage
Verification	validateCredential probe + reseller chat-completions fallback	Same flow via browser fetch

Where keys and data travel

Network destinations are clean. Photo scans, assistant chat, label and receipt parsing go only to the configured provider endpoint or the user's own reseller base URL. Barcode lookups query USDA (bundled) and OpenFoodFacts over the network — neither carries the user's key. The web-lookup tool rides the provider chat endpoint. The two exceptions found: the Google key appears in the URL query string on chat and correction paths (P2-12, header already used on the scan path), and the cross-provider chat fallback can spend a second provider's key after a primary failure (P2-8). withBaseUrl rewrites only https://api.openai.com/v1 prefixes, so a reseller URL cannot silently redirect Anthropic or Google traffic. JPEG uploads are stripped of APP1-15 and COM segments with JFIF APP0 retained, then byte-level re-validated before a payload can enter any request; the local draft file keeps its original EXIF until reset() deletes it, and never uploads.

Reseller economics

Reseller-specific exposure: the validate flow can fall back to a one-token chat-completions probe, which is a deliberate, tiny, billed call; the sanitized-schema scan path (added after the pizza bug) avoids the larger wasted retries. The remaining economics problem is the reseller's own accounting — aicredits bills each scan's image payload as prompt tokens, roughly Rs 0.49 per scan on gpt-4o-mini versus their advertised rate, about ten times more. The app cannot fix the gateway's meter, but the computeScanCost ledger should stop reporting 0 for custom models (P2-9) so users can reconcile what they are actually charged, and the per-scan caption showing model and host makes the choice visible.

8. Performance and Size Audit

Bundle and corpora

The web export is 14 MB on disk, of which the entry JavaScript bundle is 3.34 MB raw (pre-compression; CDN gzip brings the transfer to roughly a third of that). The two nutrition corpora total 6.5 MB — nutrition.db 6.21 MB with 7,930 USDA foods, ifct.db 0.31 MB with 542 rows — and are loaded lazily per screen via promise-singleton openNutritionDb(), never at root: the right architecture. Dead weight is concrete and removable: 2.5 MB of unused wasm binaries plus a debug page in public/ and four unused dependency entries (P2-32).

Artifact	Size	Assessment
Web dist (total)	14 MB	Includes 2.5 MB dead wasm + debug page (P2-32)
Entry JS bundle	3.34 MB raw	Pre-compression; minified single chunk
nutrition.db	6.21 MB	7,930 USDA foods, all energy non-NULL; lazy-loaded per screen
ifct.db	0.31 MB	542 rows, 0 NULL energy; lazy-loaded
public/ dead wasm	2.5 MB + test-wasm.html	Zero imports - remove

Boot and runtime

Cold start does little JS before first paint (only an async kv read for the onboarding gate), but the first Home paint waits on migrate plus exercise seeding that re-runs every launch — the reported slow web boot lives here along with the wasm fetch (P2-36). The heaviest runtime patterns are the one-second full-database poll in ActiveWorkout (P2-22), unbounded queries on the hottest screen (P2-34), the never-called undo-journal compaction (P2-33), and non-virtualized lists of up to 500 rows (P2-23). Each has a targeted fix in the roadmap; none require architectural change.

Token economy

Token economy is in good shape after the 54263d9 work: assistant replay is capped at 16 turns (40 on disk), today-context rides only the current message, and no food database is serialized into any prompt — verified. The remaining waste is the ~4.5K-token scan system prompt sent byte-identical on every scan with no Anthropic cache_control (P3-8), worth roughly \$0.0007 per scan at gpt-4o-mini prices, and the wire schema's inherent 2.5-3.5K tokens per structured scan.

9. Bloat Census

Bloat here means anything shipped, tracked or maintained that no code path uses. The census verified each entry by import grep or git ls-files; removals are all effort-S and are batched in roadmap Wave 1.

Dead dependencies and artifacts

Item	Evidence	Action
sql.js, wa-sqlite deps	Zero imports; superseded by @sqlite.org/sqlite-wasm	Remove
@expo/ui, expo-linking	Zero imports	Remove
public/ sqlite3.wasm, wa-sqlite*.wasm, test-wasm.html	2.5 MB dead weight + debug page	Remove

Item	Evidence	Action
src/data/backup-core.js + .js.map + .d.ts.map	Compiled artifacts git-tracked beside the .ts	gitignore
apps/mobile/patch-alert2.js	Codemod that mutates a component in place if re-run	Delete
apps/mobile/check-journal.js	Throwaway Playwright probe spawning npx serve	Delete
apps/mobile/ui-success.png	Stray screenshot	Delete
One-off scripts (report-before-after, fix-incoherent-yields, add-missing-fillings)	Already-applied patches beside the live check gates	Move to scripts/oneoff/
Dead exports: isUpToDate, latestUndoableMealOperation	Referenced nowhere outside their files	Delete or wire
Phantom zod dep in prompt tests; recipe-engine devDeps	Hoisting-dependent; vitest ^1 vs root ^3	Align

God files and duplication

Eight files exceed 690 lines. The riskiest is client.ts, which mixes three providers, three transport modes and the fallback logic in one 1,170-line module with 263-line functions - it is where both streaming P1s live, which is not a coincidence. Functions over 100 lines also include runWebLookup (101), runAssistantChatApiSingle (101), analyze (119), startReceiptScan (119), startBarcodeScan (107), AddIngredientSheet (118), dish-composer load (153), assistant send (113) and curate-drafts main (311).

File	Lines	Note
tools/indian-dishes/curated-definitions.mjs	1744	Data seed, acceptable but unwieldy
tools/indian-dishes/curate-drafts.mjs	1095	Contains a 311-line main()
src/inference/pathA/client.ts	1170	3 providers x 3 transports + fallbacks — split (Wave 4)
app/food-search.tsx	1130	Largest screen; extractable sheets
app/result.tsx	1091	AddIngredientSheet is 118 lines inline
packages/db-adapter/src/schema.ts	795	Schema + migrations
src/data/repo.ts	780	Write/undo layer; test suite pending
src/scan/orchestrator.ts	763	Four 100+-line scan functions

Duplicated logic inventory: the ingredient mini-form and the fat/method chip rows (food-search vs dish-composer), six local Field wrappers, resolveCorpusSelection twice in one file, describeOperation twice across files, one proposal-card component wearing two names, three clamp implementations, and normalizeText re-implemented across resolver layers. Console noise is limited to five sites, two of which warn on expected first-run paths. The TODO census is effectively clean, and commented-out code blocks

over ten lines do not exist - both are genuine strengths worth protecting with the ESLint rules the token file already promises.

10. Fix Roadmap

The roadmap orders all 101 findings into four waves by dependency and payoff, not by severity alone: cheap unblocking fixes first, then the reseller reliability batch that repairs the flagship user class, then the trust-and-consistency batch, then structural work that needs a dedicated session. Wave 1 is roughly one working day end to end.

Wave 1 - Quick wins (about one working day)

Fix	Findings	Effort
Pass customProviderBaseUrl to barcode and 'Other...' lookups	P1-1	S
Fix non-SSE double-append in the stream reader	P1-2	S
Wire stallTimeoutMs: 45000 into the assistant stream call	P1-3	S
Add root ErrorBoundary + per-row parse guard in Train	P1-6	S
Make saved-foods 'Log again' a working Pressable	P1-5	S
Auto-scroll the chat on content size change	P1-7	S
Surface Log-it / save failures with retry (useAction everywhere)	P1-8, P2-26, P2-27	S
Fix the literal backslash-n in correction prompt and Google join	P2-13	S
Add the two promised ESLint rules (hex literals, tap targets)	P2-20	S
Delete debris: patch-alert2.js, check-journal.js, ui-success.png, backup-core.js artifacts	P2-31, P3	S

Wave 2 - Reseller reliability and honest errors

Fix	Findings	Effort
Normalize base URLs (scheme, /v1) + reseller 404 hint	P2-11	S
Split error taxonomy: offline vs internal vs storage vs schema	P2-1, P2-2, P3-1	S
Fence-stripping in extractPayload + correction path	P2-10, P2-3	S
Thread the cancelled flag so aborts do not re-request	P2-6	S
Scan epochs + userEditedAt guards against auto-apply races	P2-4, P2-5, P2-7	M

Fix	Findings	Effort
Google key to header on chat/correction paths	P2-12	S
Honest cost for custom models (null + flag, not 0)	P2-9, P3-7	S
Same-provider-only fallback or 'answered via' disclosure + opt-in	P2-8	M

Wave 3 - Trust and consistency

Fix	Findings	Effort
Theme token batch: Screen red, Tailwind leftovers, #ccc, onboarding chrome	P2-17..19	S
Accessibility batch: chips, icon buttons, disclosures, tap targets	P2-24, P2-25, P3-29	M
Web dirty-guards (beforeunload) for custom-food and recipes	P2-15, P3-47	M
Replace DayTimeline's window.confirm shim with the shared one	P2-16	S
Empty states, copy pass, capitalization, emoji-to-Icon swap	P2-29, P3-14..16	S
Composer per-component ingredient state	P2-21	M
Result screen: Discard affordance, controlled grams, lookup-failure row	P2-14, P2-5	S

Wave 4 - Structural

Fix	Findings	Effort
Split client.ts into per-provider transports + shared SSE core	P2-3, P3-2..3	L
vitest suite for repo.ts invariants (logMeal, undo, idempotency)	P1-9	M
orchestrator decision-logic extraction + contract tests	P2-37	M
Wire compactHistory; bound streak and custom-foods queries; fix N+1	P2-33, P2-34, P2-43	S
Boot: seed short-circuit + Home skeleton	P2-36	M
Virtualize indian-dishes and train history; 1s poll only during rest	P2-22, P2-23	S
Bundle diet: remove 4 dead deps + public/ wasm; verify expo-crypto	P2-32, P3-45	S
Anthropic cache_control on the scan system prompt	P3-8	S
Corpus honesty: verified-yield metric in CI + in-app; Avial override; suggestion mechanism decision	P1-10, P2-38, P2-39	M
Web beforeunload + FTS5 fail-fast; golden tests for prompts; fixme e2e triage	P2-35, P2-41, P3-31	M

Acknowledged-but-unfixed debts from the engineering worklog, for the record: the aicredits billing inflation (outside app control, monitor via the cost ledger); push credentials living outside the sandbox (manual PAT handoff per release); the sandbox reaper killing the preview server between sessions (operational, not an app bug); phases 7 (sync), 8 (reports/health) and 11 (release) of the implementation plan still unstarted; voice mode, drag-and-drop and progress photos explicitly deferred by the owner; and the e2e suite running only in CI, not in the local check gate (P2-41).

Verification gates for every wave: npm run check (733 tests must stay green), one manual scan per mode through the aicredits gateway after Wave 2, a dark-mode and web-browser pass after Wave 3, and bundle size re-measured after Wave 4. The report's findings are the checklist; the waves are the order.